

Introduction to the Theory of Computation

Lecture 4. Finite Automata

Neo (Yuan) Li

Sep 2026

Fudan University

yuan_li@fudan.edu.cn

Why Study Finite Automata? (1/2)

The finite automaton is the simplest computation model — one *without memory*.

Reason 1: It is useful

- Regular expressions: pattern matching and text search
- Control systems: automatic doors, vending machines, elevators

Why Study Finite Automata? (2/2)

Reason 2: A stepping stone to advanced topics

Slightly extend the definition of a finite automaton, and you get a full-fledged computer — the **Turing machine**.

- Paves the way to Turing machines, nondeterministic Turing machines, decidability

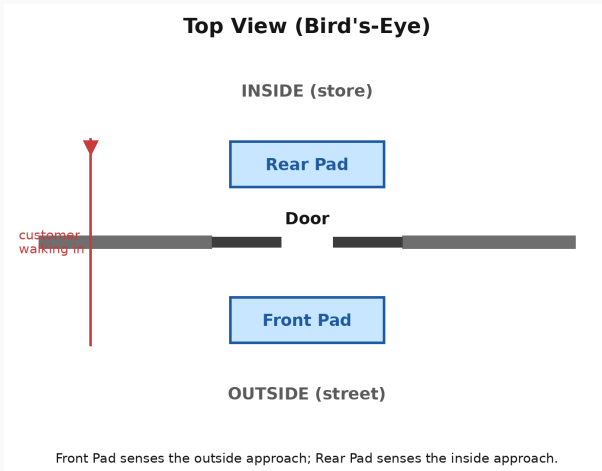
Hidden gem: what's the "slight extension"? Think about it.

Example: Sliding Door



Sensor Layout (Top View)

There are two sensors: one in the **front** (outside) and one in the **rear** (inside). The door opens if either sensor detects someone.



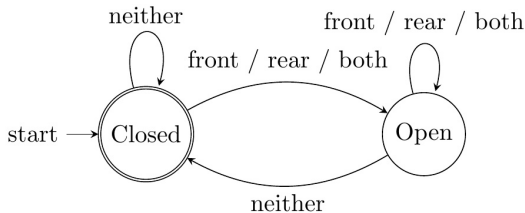
Control Rule

Given the current door state and the sensor reading, what should the door do next?

Sensor state \ Door state	Closed	Open
Front only	Open	Open
Rear only	Open	Open
Both	Open	Open
Neither	Closed	Closed

State Transition Diagram

We can visualize this control rule as a finite automaton with two states.



Exercise: One-Way Entrance Gate

What if we want a one-way entrance?

Design a one-way entrance gate using a finite automaton.

Allow people to **enter**, but **not exit**.

- Same two sensors as before: **front** and **rear**
- Think about: how many states do you need? What should each transition be?

Example: Divisibility by 2

Consider the language

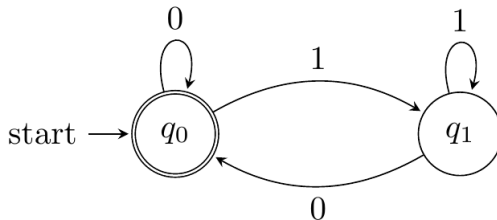
$$L = \{w \in \{0,1\}^+ : w = w_1w_2 \cdots w_n, w_n = 0\}.$$

In other words, L is the set of all binary strings representing an **even number**, written with the **most significant bit first (MSB)**.

Question: Can a FA decide this language?

A DFA for L

The following DFA accepts the language L :



Try It Yourself: Online DFA Simulator

Run this DFA online:

[*https://web.cs.ucdavis.edu/~doty/automata/*](https://web.cs.ucdavis.edu/~doty/automata/)

Select **DFA**, then paste in the following code:

```
states: [q0, q1]
input_alphabet: [0, 1]
start_state: q0
accept_states: [q0]
delta:
  q0:
    0: q0
    1: q1
  q1:
    0: q0
    1: q1
```

Example: Divisibility by 3

Consider the language

$$L = \{w \in \{0, 1\}^* : w = w_1w_2 \cdots w_n,$$

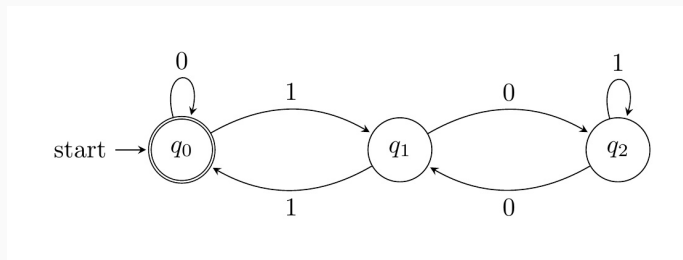
$$\sum_{i=1}^n w_i 2^{n-i} \equiv 0 \pmod{3}\}.$$

In other words, L is the set of all binary strings representing a **multiple of 3**, written with the **most significant bit first (MSB)**.

Question: Can a FA decide this language?

A DFA for L

The following DFA accepts the language L :



For example, the DFA accepts 0, 11, 1001, etc.

Try It Yourself: Online DFA Simulator

Open the simulator:

[`https://web.cs.ucdavis.edu/~doty/automata/`](https://web.cs.ucdavis.edu/~doty/automata/)

Select **DFA**, and try to build the divisibility-by-3 DFA yourself:

- 3 states, representing remainders 0, 1, 2 (mod 3)
- Input alphabet: 0, 1
- Use the transition table from before to fill in *delta*

Test it on a few strings — does it agree with your expectations?

The DFA Code

```
states: [q0, q1, q2]
input_alphabet: [0, 1]
start_state: q0
accept_states: [q0]
delta:
  q0:
    0: q0
    1: q1
  q1:
    0: q2
    1: q0
  q2:
    0: q1
    1: q2
```

In-Class Exercise: Design a Finite Automaton

Design a finite automaton for each of the following languages.

1. $L = \{ w \in \{0, 1\}^* :$
the number of 1's in w is a multiple of 3 }
2. $L = \{ w \in \{0, 1\}^* : w \text{ starts with } 01 \}$
3. $L = \{ w \in \{0, 1\}^* : w \text{ ends with } 00 \}$

Definition: Finite Automaton

Definition 1

A finite automaton is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*.
2. Σ is a finite set called the *alphabet*.
3. $\delta : Q \times \Sigma \rightarrow Q$ is the *transition function*.
4. $q_0 \in Q$ is the *start state*.
5. $F \subseteq Q$ is the set of *accept states*.

Formalizing Example: Divisibility by 2

Recall the DFA from before. We have

$$\Sigma = \{0, 1\}, \quad Q = \{q_0, q_1\}.$$

The transition function δ is given by the table:

	0	1
q_0	q_0	q_1
q_1	q_0	q_1

The start state is $q_0 \in Q$, and the set of accept states is $F = \{q_0\}$.

Formalizing Example: Divisibility by 3

Recall the DFA from before. We have

$$\Sigma = \{0, 1\}, \quad Q = \{q_0, q_1, q_2\}.$$

The transition function δ is given by the table:

	0	1
q_0	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2

The start state is $q_0 \in Q$, and the set of accept states is $F = \{q_0\}$.

Definition: A Finite Automaton Accepting a String

Definition 2

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton. Let $w = w_1w_2 \cdots w_n$ be a string where each $w_i \in \Sigma$.

We say M *accepts* w if there exists a sequence of states $r_0, r_1, \dots, r_n \in Q$ such that

1. $r_0 = q_0$,
2. $\delta(r_i, w_{i+1}) = r_{i+1}$, for $i = 0, 1, \dots, n - 1$,
3. $r_n \in F$.

Definition 3

The *language accepted by M* , denoted $L(M)$, is the set of all strings M accepts. We say M *recognizes* (or *accepts*) $L(M)$.